

Backend de la plataforma Mujer Marinilla

Requisitos e Historias de Usuario para el equipo de practicantes de desarrollo

Plataforma web de orientación a mujeres víctimas de violencia en el municipio de **Marinilla, Antioquia**, adaptada del programa «*Mulher Tem Saída*» de Río de Janeiro. Este documento especifica el alcance, los requisitos y las historias de usuario del **Producto Mínimo Viable (MVP)** del backend, a construir en **Python (FastAPI)**, incluido el **modelado de la base de datos**.

Proyecto	Backend · Mujer Marinilla
Documento	Especificación de requisitos e historias de usuario — MVP
Versión	1.1
Fecha	13 de agosto de 2026
Dirigido a	Equipo de practicantes de desarrollo
Stack definido	Python · FastAPI · PostgreSQL · JWT
Estado	Aprobado para desarrollo

Documento confidencial de trabajo. Dominio sensible: plataforma de atención a víctimas de violencia. Léase la sección 4 (Restricciones de privacidad) antes de escribir código.

Contenido

1. **1.** Introducción y objetivo
2. **2.** Alcance del MVP
3. **3.** Contexto del producto
4. **4.** Restricciones de privacidad (crítico)
5. **5.** Stack tecnológico
6. **6.** Requisitos funcionales
7. **7.** Requisitos no funcionales
8. **8.** Dominio de datos (referencia para el modelado)
9. **9.** Historias de usuario
10. **10.** Definición de Terminado (DoD)
11. **11.** Plan por sprints
12. **12.** Glosario y preguntas abiertas

1. Introducción y objetivo

La aplicación **Mujer Marinilla** es una plataforma web que permite a mujeres víctimas de violencia encontrar servicios de atención geolocalizados y consultar líneas de ayuda. Hoy el frontend (Next.js) es **100% estático**: todos los datos están escritos directamente en archivos de código (`src/data/`). Cambiar un teléfono o agregar una unidad exige editar código y volver a desplegar.

Objetivo del MVP del backend: diseñar la base de datos y exponer esos datos mediante una API REST para que:

1. El personal autorizado de la Alcaldía pueda **actualizar los datos sin depender de un desarrollador**.
2. El frontend **consume los datos por API** en tiempo de ejecución en lugar de tenerlos incrustados.

El **modelado de la base de datos es parte del encargo**: el equipo debe diseñar el modelo relacional a partir del dominio descrito en este documento.

2. Alcance del MVP

2.1 Incluido

- **Diseño y modelado de la base de datos** (diagrama entidad-relación, tipos, llaves y relaciones).
- API REST de **lectura pública** para los maestros de datos.
- Operaciones de **escritura protegidas** (crear/editar/eliminar) para administradores.
- **Autenticación de administradores** con token JWT.

- Carga inicial de datos (*seed*) desde el contenido actual del frontend.
- Documentación automática de la API (Swagger/OpenAPI).

2.2 Fuera de alcance (esta etapa)

- Panel de administración con interfaz gráfica (la administración la consumirá el frontend / Swagger).
- Recolección de datos del formulario de onboarding o de cualquier dato personal de las usuarias.
- Métricas de uso, incluso anónimas.
- Notificaciones, mensajería o integración con terceros.

3. Contexto del producto

El frontend ya existe y define la forma de los datos. El modelo de datos del backend debe **reflejar el mismo dominio** para que la migración sea transparente. Fuentes de verdad actuales:

Entidad	Archivo actual (frontend)	Rol
Unidad / Equipamiento	<code>src/data/equipments.ts</code>	Servicios de atención en el mapa
Línea telefónica	<code>src/data/phones.ts</code>	Líneas de ayuda
Tipo de violencia	<code>src/data/violenceTypes.ts</code>	Glosario educativo
Tipo de unidad	<code>src/data/equipmentsTypes.ts</code>	Catálogo/clasificación
Ruta de atención	<code>src/data/encaminhamento.ts</code>	Orientación según el caso
Festivos	<code>src/utils/holidays.ts</code>	Cálculo de horarios de apertura

4. Restricciones de privacidad (crítico · no negociable)

⚠ SEGURIDAD DE LAS USUARIAS

La aplicación promete a las usuarias que «**las respuestas son anónimas**». El incumplimiento de estas reglas pone en riesgo la vida de las víctimas. Son requisitos obligatorios, no preferencias técnicas.

- **Prohibido** almacenar datos personales o identificables de las usuarias: respuestas del formulario asociadas a una persona, dirección IP, geolocalización individual o identificadores de dispositivo.
- El **formulario de onboarding no envía nada al backend**: enruta únicamente en el navegador. No debe modelarse ninguna tabla ni endpoint que lo reciba.
- Ningún endpoint del MVP recibe ni persiste datos de las usuarias finales.

- Los únicos datos personales del sistema son las **credenciales de los administradores**, que se almacenan con la contraseña *hasheada* (bcrypt), nunca en texto plano.

5. Stack tecnológico

Capa	Tecnología	Notas
Framework	FastAPI	Async, validación Pydantic, Swagger automático
Servidor	Uvicorn	ASGI
Validación/esquemas	Pydantic v2	Alias camelCase para el frontend
ORM	SQLAlchemy 2.0	Mapea el modelo de datos diseñado por el equipo
Migraciones	Alembic	Versionado del esquema
Base de datos	PostgreSQL (prod) / SQLite (dev)	SQLite sin configuración para desarrollo
Autenticación	OAuth2 Password + JWT	<code>python-jose</code> , <code>passlib[bcrypt]</code>
Modelado	Diagrama ER (dbdiagram.io / draw.io)	Entregable de diseño previo a la implementación
Pruebas	pytest + httpx	Cobertura mínima: 1 CRUD + auth
Contenedor	Docker	Despliegue en GCP (alineado al frontend)

6. Requisitos funcionales

ID	Requisito	Prioridad
RF-01	El equipo diseña y documenta el modelo de datos (diagrama ER) que soporta las seis entidades del dominio.	Must
RF-02	El sistema expone endpoints de lectura pública (lista y detalle) para las seis entidades de datos.	Must
RF-03	El sistema permite crear, editar y eliminar registros de cada entidad, solo a administradores autenticados.	Must
RF-04	Un administrador puede iniciar sesión con correo y contraseña y recibir un token JWT.	Must
RF-05	Todo endpoint de escritura rechaza (401) las peticiones sin token válido.	Must
RF-06	El endpoint de unidades admite filtros por tipo de equipamiento y, opcionalmente, por <i>bounding box</i> geográfico.	Should
RF-07	El sistema provee un proceso de carga inicial (seed) que reproduce los datos actuales del frontend.	Must
RF-08	El sistema publica documentación OpenAPI navegable en <code>/docs</code> .	Must
RF-09	El sistema expone un endpoint <code>/health</code> para verificación de estado.	Must
RF-10	El primer administrador se crea mediante comando de consola (no existe registro público).	Must
RF-11	Las respuestas usan el mismo formato de campos que las interfaces del frontend (camelCase).	Should

7. Requisitos no funcionales

ID	Requisito
RNF-01 · Privacidad	Ningún dato personal de usuarias finales se recibe ni almacena (ver sección 4).

RNF-02 · Seguridad	Contraseñas con hash bcrypt; secretos (JWT, BD) por variables de entorno, nunca en el repositorio.
RNF-03 · Interoperabilidad	CORS habilitado solo para los orígenes del frontend (dev y producción).
RNF-04 · Mantenibilidad	Modelo de datos normalizado y documentado (diagrama ER); código por capas (models / schemas / crud / routers); esquema versionado con Alembic.
RNF-05 · Portabilidad	Ejecutable localmente con SQLite y en producción con PostgreSQL sin cambios de código.
RNF-06 · Observabilidad	Endpoint <code>/health</code> y registro (logs) de errores de servidor.
RNF-07 · Documentación	README con pasos de instalación, ejecución, seed y despliegue; Swagger operativo.
RNF-08 · Calidad	Pruebas automatizadas de al menos un CRUD completo y del flujo de autenticación.

8. Dominio de datos (referencia para el modelado)

CÓMO LEER ESTA SECCIÓN

Lo siguiente describe el **dominio** (qué información existe y cómo la usa el frontend). **El diseño del modelo relacional es responsabilidad del equipo** (ver HU-01): tablas, tipos de datos, llaves primarias y foráneas, restricciones, normalización y manejo de catálogos y campos de lista. Los campos con `?` son opcionales; los nombres de columna quedan a criterio del equipo (snake_case en BD), pero la **API debe exponer camelCase**.

8.1 Unidad / Equipamiento (Equipment)

`id`, `equipmentType` (Salud | Servicios para la Mujer | Estación de Policía), `type?`, `name`, `abbreviation?`, `openingTime?`, `closingTime?`, `phones?[]`, `description?`, `function?`, `violenceTypes?`, `address?`, `neighborhood?`, `notes?`, `latitude?`, `longitude?`, `openingTimeSaturday?`, `closingTimeSaturday?`, `openingTimeSunday?`, `closingTimeSunday?`, `workingDates?[]`, `open24` (0|1).

8.2 Línea telefónica (Phone)

`id`, `number`, `title`, `description`.

8.3 Tipo de violencia (ViolenceType)

`id`, `name`, `description`, `examples?[]`, `image?`.

8.4 Tipo de unidad — catálogo (EquipmentType)

`id`, `abbreviation`, `name`, `description`.

8.5 Ruta de atención (Referral)

`id`, `header`, `text`, `image?`, `name?`, `idTipoViolencia` (relación con Tipo de violencia).

8.6 Festivo (Holiday)

`id`, `date`, `name`. (Reemplazar festivos de Brasil por los de Colombia + municipales de Marinilla.)

8.7 Administrador (AdminUser) — interno, no expuesto en lecturas

`id`, `email` (único), `hashedPassword`, `isActive`, `createdAt`.

DECISIONES DE MODELADO A TOMAR POR EL EQUIPO

- Los tipos de unidad hoy usan una taxonomía brasileña (CEAM, NEAM...), pendiente de adaptar a Marinilla. Conviene modelarlos como **catálogo/tabla** configurable y relacionar Unidad → Tipo de unidad, en lugar de valores fijos en código.
- Los campos de lista (`phones` , `workingDates` , `examples`): decidir entre columna JSON/array o tabla relacionada, y justificarlo.
- La relación Ruta de atención → Tipo de violencia debe modelarse con **llave foránea**.
- Definir tipos adecuados para horarios y coordenadas, y las restricciones (NOT NULL, únicos).

9. Historias de usuario

Roles: **Administrador** (funcionario autorizado de la Alcaldía), **Aplicación cliente** (el frontend que consume la API) y **Desarrollador** (el equipo, para historias técnicas y de diseño). Prioridad en notación MoSCoW; estimación en puntos de historia.

Épica 1 — Diseño y andamiaje · modelado e infraestructura base

HU-01 Modelado de la base de datos

MUST

5 PTS

Como desarrollador, **quiero** diseñar y documentar el modelo de datos (entidades, atributos, tipos, llaves y relaciones) a partir del dominio, **para** contar con una base de datos normalizada y coherente antes de implementar.

CRITERIOS DE ACEPTACIÓN

- **Dado** el dominio de la sección 8, **cuando** modelo la BD, **entonces** entrego un **diagrama entidad-relación** con todas las entidades, atributos y relaciones.
- **Entonces** cada tabla define su llave primaria, tipos de datos y restricciones (NOT NULL, únicos) apropiados.
- **Entonces** las relaciones (p. ej. Ruta de atención → Tipo de violencia; Unidad → Tipo de unidad) se modelan con llaves foráneas.
- **Entonces** se documenta y justifica el manejo de los campos de lista y de los catálogos de tipos.
- **Entonces** el modelo se refleja fielmente en los modelos SQLAlchemy y en las migraciones de Alembic (HU-03).

HU-02 Proyecto base con healthcheck y documentación

MUST

3 PTS

Como desarrollador, **quiero** un proyecto FastAPI ejecutable con endpoint de salud y documentación automática, **para** tener una base de trabajo verificable.

CRITERIOS DE ACEPTACIÓN

- **Dado** el proyecto instalado, **cuando** ejecuto el servidor, **entonces** `GET /health` responde `200` con `{"status": "ok"}`.
- **Cuando** abro `/docs`, **entonces** veo la interfaz Swagger de la API.
- **Entonces** el README explica cómo instalar y ejecutar el proyecto.

HU-03 Persistencia con base de datos y migraciones

MUST

5 PTS

Como desarrollador, quiero implementar el modelo (HU-01) con SQLAlchemy y migraciones Alembic, **para** persistir los datos de forma versionada y reproducible.

CRITERIOS DE ACEPTACIÓN

- **Dado** `DATABASE_URL` configurado, **cuando** arranca la app, **entonces** conecta a SQLite (dev) o PostgreSQL (prod) sin cambios de código.
- **Cuando** ejecuto las migraciones, **entonces** se crean todas las tablas del modelo diseñado en HU-01.
- **Entonces** existe una sesión de BD inyectable por dependencia (`get_db`).

Épica 2 — Gestión de maestros de datos (CRUD) · corazón del MVP

HU-04 Consultar unidades de atención

MUST

5 PTS

Como aplicación cliente, quiero listar y consultar el detalle de las unidades de atención, **para** mostrarlas en el mapa y en las fichas.

CRITERIOS DE ACEPTACIÓN

- **Cuando** hago `GET /api/v1/equipments`, **entonces** recibo la lista con la misma forma de campos que el frontend (camelCase).
- **Cuando** hago `GET /api/v1/equipments/{id}` de un id existente, **entonces** recibo su detalle; si no existe, `404`.
- **Entonces** los campos de lista (`phones`, `workingDates`) se devuelven como arreglos JSON.
- **Entonces** el endpoint es público (no requiere token).

HU-05 Administrar unidades de atención

MUST

5 PTS

Como administrador, quiero crear, editar y eliminar unidades, **para** mantener el inventario de servicios actualizado sin depender de un desarrollador.

CRITERIOS DE ACEPTACIÓN

- **Dado** un token de admin válido, **cuando** hago `POST` con datos válidos, **entonces** la unidad se crea y se devuelve con su `id` (`201`).
- **Cuando** hago `PUT/PATCH` sobre un id existente, **entonces** los cambios se persisten.
- **Cuando** hago `DELETE`, **entonces** la unidad deja de aparecer en las lecturas.
- **Cuando** envío datos inválidos, **entonces** recibo `422` con el detalle de validación.
- **Cuando** no envío token, **entonces** recibo `401`.

HU-06 CRUD del resto de maestros de datos

MUST

8 PTS

Como aplicación cliente y administrador, **quiero** las mismas operaciones de consulta y administración para **teléfonos, tipos de violencia, tipos de unidad, ruta de atención y festivos**, para **gestionar** todo el contenido del sitio.

CRITERIOS DE ACEPTACIÓN

- **Entonces** cada entidad expone el mismo patrón que HU-04/HU-05: lecturas públicas, escrituras protegidas.
- **Entonces** las rutas siguen el patrón `/api/v1/phones` , `/api/v1/violence-types` , `/api/v1/equipment-types` , `/api/v1/referrals` , `/api/v1/holidays` .
- **Entonces** la forma de los campos coincide con el dominio (sección 8).

HU-07 Filtrar unidades para el mapa

SHOULD

3 PTS

Como aplicación cliente, **quiero** filtrar las unidades por tipo y por área geográfica, **para** mostrar solo las relevantes al usuario en el mapa.

CRITERIOS DE ACEPTACIÓN

- **Cuando** uso `?equipmentType=...` , **entonces** recibo solo las unidades de ese tipo.
- **Cuando** uso `?bbox=minLng,minLat,maxLng,maxLat` , **entonces** recibo solo las unidades dentro de esa área.
- **Cuando** combino filtros, **entonces** se aplican de forma acumulativa.

HU-08 Carga inicial de datos (seed)

MUST

5 PTS

Como desarrollador, **quiero** un proceso de carga inicial con los datos actuales del frontend, **para** poblar la base de datos sin captura manual.

CRITERIOS DE ACEPTACIÓN

- **Cuando** ejecuto el comando de seed sobre una BD vacía, **entonces** se cargan las seis entidades con los datos de `src/data/` .
- **Cuando** lo ejecuto de nuevo, **entonces** no se duplican registros (idempotente).

HU-09 Inicio de sesión de administrador

MUST

5 PTS

Como administrador, quiero iniciar sesión con mi correo y contraseña y recibir un token, **para** acceder a las funciones de administración.

CRITERIOS DE ACEPTACIÓN

- **Dado** un admin registrado, **cuando** hago `POST /api/v1/auth/login` con credenciales correctas, **entonces** recibo un `access_token` JWT.
- **Cuando** las credenciales son incorrectas, **entonces** recibo `401` sin revelar cuál dato falló.
- **Entonces** la contraseña se almacena con hash bcrypt, nunca en texto plano.
- **Cuando** consulto `GET /api/v1/auth/me` con el token, **entonces** recibo los datos del admin autenticado.

HU-10 Protección de operaciones de escritura

MUST

3 PTS

Como responsable del sistema, quiero que solo los administradores autenticados puedan modificar datos, **para** garantizar la integridad de la información.

CRITERIOS DE ACEPTACIÓN

- **Cuando** se llama cualquier `POST/PUT/PATCH/DELETE` sin token, **entonces** la respuesta es `401`.
- **Cuando** el token es inválido o expiró, **entonces** la respuesta es `401`.
- **Entonces** las lecturas (`GET`) siguen siendo públicas.

HU-11 Creación del primer administrador

MUST

2 PTS

Como responsable del despliegue, quiero crear el primer administrador por consola, **para** no exponer un registro público de cuentas.

CRITERIOS DE ACEPTACIÓN

- **Cuando** ejecuto el comando de creación con correo y contraseña, **entonces** se crea un admin activo con la contraseña *hasheada*.
- **Entonces** no existe ningún endpoint público de registro de administradores.

HU-12 Consumo desde el frontend (CORS y formato)

SHOULD

3 PTS

Como aplicación cliente, **quiero** consumir la API desde el dominio del frontend con el formato de campos esperado, **para** integrarme sin cambiar los tipos existentes.

CRITERIOS DE ACEPTACIÓN

- **Cuando** el frontend (dev o producción) llama a la API, **entonces** CORS permite el origen configurado por variable de entorno.
- **Entonces** las respuestas usan camelCase, coincidiendo con las interfaces TypeScript.

HU-13 Empaquetado y despliegue

SHOULD

5 PTS

Como equipo de operaciones, **quiero** una imagen Docker y configuración por variables de entorno, **para** desplegar el backend en GCP junto al frontend.

CRITERIOS DE ACEPTACIÓN

- **Cuando** construyo la imagen Docker, **entonces** el contenedor arranca el servicio con Unicorn.
- **Entonces** todos los secretos (JWT, credenciales de BD) se leen de variables de entorno; ninguno está en el repositorio.
- **Entonces** el README documenta las variables requeridas y el paso de migraciones previo al arranque.

10. Definición de Terminado (DoD)

Una historia se considera terminada cuando cumple **todos** estos puntos:

- Cumple sus criterios de aceptación.
- El código sigue la estructura por capas acordada y pasa el *linter*.
- Tiene pruebas automatizadas cuando aplica (mínimo: un CRUD completo y el flujo de auth).
- Los endpoints aparecen correctamente en Swagger.
- El cambio respeta el modelo de datos aprobado (HU-01) y sus migraciones.
- No introduce almacenamiento de datos personales de usuarias (revisión de la sección 4).
- No hay secretos en el repositorio.
- Revisado por al menos otra persona del equipo (*pull request* aprobado).

11. Plan por sprints

Sprint	Objetivo	Historias
1 Diseño y andamiaje	Modelo de datos (ER) aprobado, servicio en marcha, BD y migraciones, primer CRUD de punta a punta (Phone) con pruebas.	HU-01, HU-02, HU-03, parte de HU-06
2 Maestros de datos	CRUD completo de todas las entidades, filtros de unidades y seed inicial.	HU-04, HU-05, HU-06, HU-07, HU-08
3 Seguridad e integración	Autenticación de administradores, protección de escrituras, CORS/formato, Docker y despliegue.	HU-09, HU-10, HU-11, HU-12, HU-13

12. Glosario y preguntas abiertas

12.1 Glosario

Término	Significado
MVP	Producto Mínimo Viable: la versión más pequeña que aporta valor y se puede usar.
HU	Historia de Usuario: descripción breve de una funcionalidad desde la perspectiva de quien la usa.
CRUD	Crear, Leer, Actualizar, Eliminar (operaciones básicas sobre datos).

Modelo ER	Diagrama Entidad-Relación: representación de las tablas, sus atributos y las relaciones entre ellas.
JWT	Token firmado que acredita la identidad del administrador en cada petición.
Seed	Carga inicial de datos en la base de datos.
MoSCoW	Priorización: Must (obligatorio), Should (deseable), Could, Won't.

12.2 Preguntas abiertas (a resolver con la Alcaldía)

- Taxonomía definitiva de tipos de unidad en Marinilla (reemplaza la clasificación brasileña).
- ¿Habrà más de un administrador? ¿Se requieren roles diferenciados?
- Dominio de producción del frontend (para configurar CORS).
- Formato de la ruta de atención (`referrals`): ¿texto plano, HTML o enlaces?
- Festivos oficiales de Colombia y municipales de Marinilla a cargar.